

1. Problem Statement and Proposed Solution

Problem Description

Modern messaging platforms suffer from chronic information overload. As fast-paced group chats and multi-user threads cascade continuously, users who step away for just a few hours are forced to sift through hundreds of unorganized messages. Important action items, critical updates, and key decisions inevitably get buried in conversational noise.

- **Catch-Up Overhead:** The average user spends an estimated 15 to 20 minutes daily simply scrolling backward to catch up on missed conversations.
- **Information Attrition:** Internal product surveys indicate that 42% of users report missing important logistical details or deadlines due to conversation volume.
- **Engagement Decay:** Users experiencing high notification volumes combined with low message readability show a 14% drop in daily active engagement over a 30-day period, often opting to mute channels permanently.

Impact Assessment

- **User Experience (UX):** Continuous scrolling induces cognitive fatigue, transforming a collaborative tool into a source of stress.
- **Retention & Engagement:** When channels are permanently muted, user retention plummets. Platform interaction shifts from proactive participation to reactive damage control.
- **Competitive Disadvantage:** In a market where competing platforms are rapidly deploying AI-driven catch-up mechanics, lacking a native, intelligent summarizing feature signals stagnation and accelerates user churn to smarter ecosystems.

Solution Vision

The proposed solution introduces an automated, real-time dialogue summarization feature.

- **Cognitive Relief:** Condenses multi-page chat transcripts into bulleted, actionable summaries, reducing catch-up time to under 30 seconds.
- **Accessibility:** Allows users to rapidly grasp context across multiple active channels.
- **Monetization & Premium Tiers:** Serves as a high-value anchor feature for premium subscription models or corporate enterprise tiers seeking productivity optimizations.

Success Criteria

To validate the feature before production deployment, the project must meet the following benchmark thresholds:

Metric Category	Target Metric / Benchmark	Business & Technical Rationale
Summarization Quality	ROUGE-1 $\geq$ 45, ROUGE-2 $\geq$ 22, ROUGE-L $\geq$ 40	Ensures semantic overlap and structural alignment with human-validated summaries.
Technical Latency	Inference Time <2.5 seconds per transcript	Crucial for real-time UI/UX responsiveness and avoiding user bottlenecks.
Resource Efficiency	Model footprint compressed via quantization ($\leq$ 8-bit)	Lowers cloud compute (GPU/vCPU) overhead during inference scaling.
User Engagement	+10% recovery in active engagement among previously muted threads	Measures successful mitigation of information overload.

2. Problem Solving Process

Process Framework

The development lifecycle is broken down into a structured, 6-step engineering pipeline:

[1. Data Prep & SAMSum EDA] → [2. Baseline & Architecture Selection] → [3. Fine-Tuning Pipeline]

[6. Deployment & Quantization] □ [5. Evaluation (ROUGE & LLM)] □ [4. Optimization Iteration]

1. Data Exploration & Preparation: Analyze the SAMSum dataset. Tokenize conversational structures, handle speaker tags (User1:), manage sequence truncation, and establish baseline text statistics.
2. Model Architecture Selection: Set up a local sequence-to-sequence baseline using a pre-trained encoder-decoder model (e.g., BART-Large or T5).
3. Training & Fine-Tuning Pipeline: Implement a supervised fine-tuning (SFT) loop using Hugging Face Transformers and PyTorch, optimizing for conditional cross-entropy loss.
4. Optimization Iteration: Integrate mixed-precision training (FP16), gradient accumulation, and learning rate scheduling to manage compute bounds.
5. Evaluation & Testing: Run automated ROUGE evaluation scoring alongside zero-shot/few-shot comparative prompting with ChatGPT (GPT-4o mini) acting as a golden-standard evaluator.

6. Deployment Considerations: Export the final model artifact to ONNX or convert it via 8-bit quantization to evaluate production readiness.

Methodology Justification

Why BERT-Based Encoder-Decoder & Auto-Regressive ChatGPT?

- The Encoder-Decoder Advantage: Dialogue features scattered, non-linear context. A bidirectional encoder (like the one found in BART) reads the entire chat history at once to capture deep context, while the auto-regressive decoder cleanly synthesizes fluid, natural human summaries.
- ChatGPT Complementarity: While fine-tuning a local encoder-decoder model gives us full data privacy and low operational latency, leveraging ChatGPT via auto-regressive prompting serves as a powerful performance ceiling benchmark and structural teacher for zero-shot comparisons.

Fine-Tuning vs. Training from Scratch

Training a transformer from scratch requires millions of conversational pairs and massive capital. Fine-tuning a pre-trained model (like facebook/bart-large-cnn) allows us to leverage pre-existing linguistic syntax and world knowledge, adapting it to dialogue structures using minimal data and computation.

Evaluation Metrics Suitability

- ROUGE (Recall-Oriented Understudy for Gisting Evaluation): Quantifies n-gram overlap. It is the objective standard for ensuring core keywords and entities transfer from the chat to the summary.
- Human/LLM Evaluation: ROUGE can penalize perfectly valid paraphrasing. Using an LLM evaluator helps score abstract traits like factual consistency, lack of hallucination, and conversational tone.

3. Timeline and Scope

This 6-week roadmap balances research, rigid iteration, and risk buffer zones.

Weeks: 01 02 03 04 05 06

[R&D]

[--- Implementation ---]

[Iteration]

[Risk/Doc]

[Delivery]

Phase 1: Research and Preparation

- Deconstruct Transformer seq-to-seq architectures, attention mechanisms, and decoding strategies (beam search vs. top-p sampling).
- Conduct exploratory data analysis (EDA) on the SAMSum dataset to calculate token length distributions.
- *Milestone:* Complete written review of baseline metrics and data ingestion scripts.

Phase 2: Implementation & Base Training

- Build data loading, preprocessing, and tokenization pipelines.
- Set up the core PyTorch/Hugging Face training loop.
- Execute baseline training runs on a standard GPU instance.
- *Milestone:* Successful completion of an initial epoch without memory fragmentation; generate baseline ROUGE scores.

Phase 3: Optimization & Evaluation Iteration

- Apply optimization techniques: Hyperparameter tuning (learning rate warming, weight decay) and model compression.
- Implement LLM-as-a-judge evaluation scripts utilizing the ChatGPT API.
- *Iteration Point:* If ROUGE scores stall, pivot from full parameter fine-tuning to Parameter-Efficient Fine-Tuning (PEFT/LoRA) to stabilize gradient steps.

Phase 4: Risk Management & Validation Buffer

- Compute Risk Mitigation: If GPU resource limits are breached, implement aggressive gradient accumulation (e.g., steps=4) and downscale batch size to prevent Out-Of-Memory (OOM) errors.
- Review edge cases (e.g., extremely long group threads) and establish safe sequence truncation lengths.

Phase 5: Documentation & Final Delivery

- Clean, modularize, and push production-ready code to the public GitHub repository.
- Author a comprehensive README.md detailing architecture choices, reproducibility steps, and optimization graphs.
- Compile the final presentation slide deck summarizing technical achievements against the business problem.